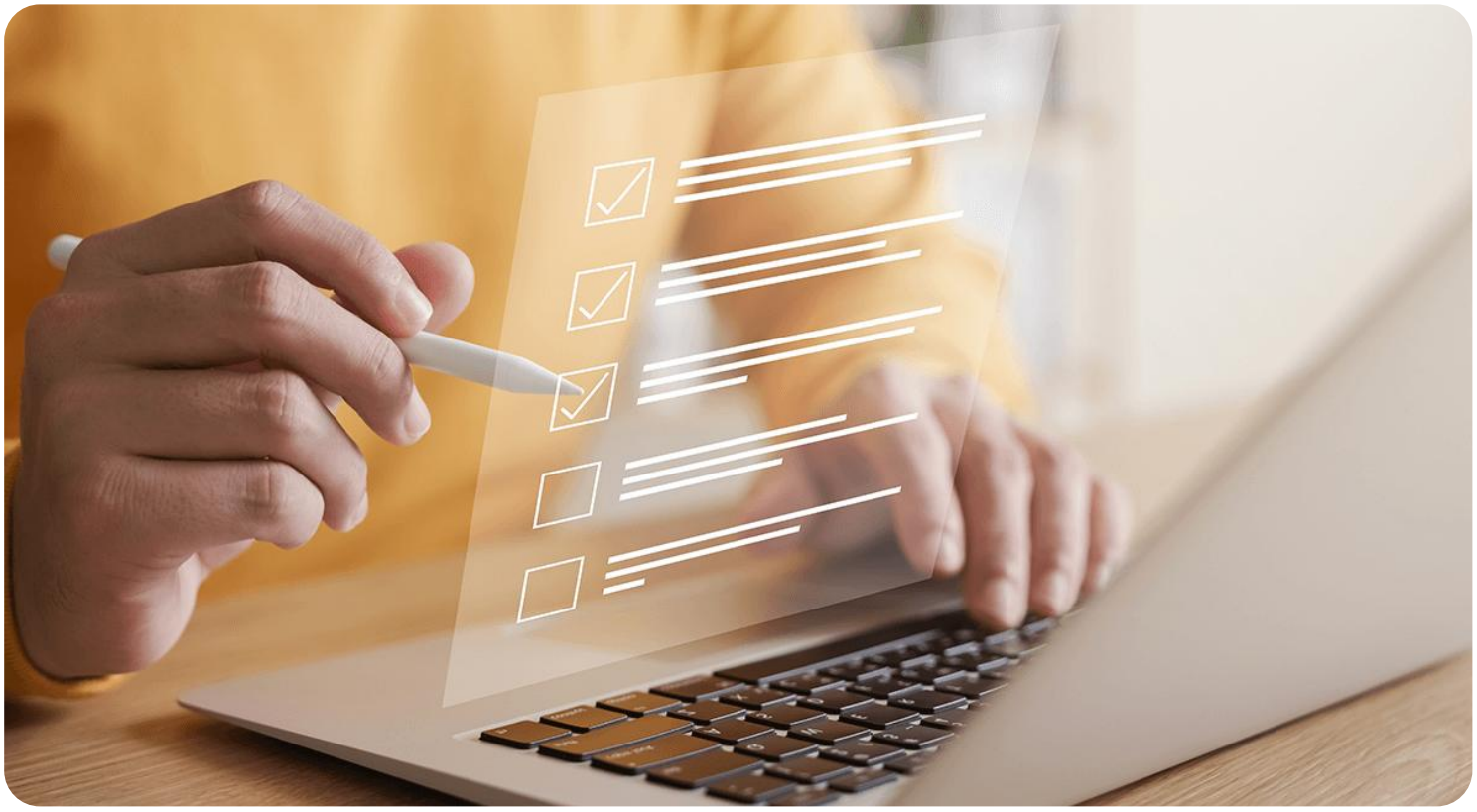
DuocUC | ONLINE



EFT – Semana 9

PBY2203 Desarrollo Backend III

Informe Técnico: modernización del sistema legacy del Banco XYZ (microservicios, BFF y procesos batch)

Nombre estudiantes:	Antonio Muñoz
Carrera: Analista Programador Computacional	
Profesor: Marcelo Zepeda	**Fecha: 11 – 10 - 2026**

Tabla de contenido

1. Resumen ejecutivo 3

2. Identificación de procesos clave del legacy 3

3. Propuesta de arquitectura 4

3.1 Requerimientos de negocio clave y cómo los resuelve la arquitectura 4

3.2 Microservicios, responsabilidades y límites 5

3.3 BFF por canal 5

4. Migración de procesos batch con Spring Batch 5

4.1 Diseño común 6

4.2 Reglas de negocio de cada job 6

4.3 Verificación de equivalencia con el legacy 6

5. Consistencia de datos, resiliencia y mensajería 7

5.1 Consistencia en un entorno distribuido 7

5.2 Resiliencia y comportamientos alternativos 8

5.3 Seguridad 9

5.4 Mensajería con Kafka 9

6. Contenerización, escalabilidad y monitorización 9

7. Comparación con el sistema legacy 10

8. Desafíos, soluciones y mejoras futuras 10

9. Conclusiones 10

1. Resumen ejecutivo

El Banco XYZ opera procesos críticos (reporte de transacciones diarias, cálculo de intereses y estados de cuenta) en un sistema legacy basado en COBOL y scripts Shell sobre mainframe. Esta solución los migra a una arquitectura moderna sobre Java 21 y Spring Boot 4: cuatro microservicios de negocio independientes (clientes, cuentas, pagos y procesos batch), tres BFF que adaptan la información a cada canal (web, móvil y cajero), una plataforma Spring Cloud (Config, Eureka, OAuth2), mensajería con Kafka y orquestación con Docker Compose.

Respecto de la entrega anterior, el antiguo servicio único de dominio («Core») se dividió en tres microservicios autónomos, cada uno con su propia base de datos, y los BFF quedaron acotados a su rol de adaptadores de interfaz. Los procesos batch se reescribieron con Spring Batch, con políticas de omisión (*skip*), reintento (*retry*) y reanudación (*restart*) frente a los datos defectuosos del legacy.

2. Identificación de procesos clave del legacy

A partir de los archivos del legacy (repositorio fin_legacy_data) y del dominio bancario, se identificaron cinco procesos, que determinan los límites de cada servicio:

#	Proceso legacy	Descripción	Destino en la nueva arquitectura
1	Reporte de transacciones diarias	Consolida débitos y créditos por día y marca anomalías (montos negativos o cero)	Job reporteTransaccionesDiariasJob (batch-service)
2	Cálculo de intereses	Interés mensual por cuenta según tipo (ahorro, préstamo, hipoteca)	Job calculoInteresesMensualesJob (batch-service)
3	Estados de cuenta anuales	Consolida movimientos por cuenta y año (depósitos, egresos, saldo neto)	Job estadosCuentaAnualesJob (batch-service)
4	Gestión de cuentas	Apertura, cierre, mantenimiento y libro de movimientos	cuentas-service
5	Procesamiento de pagos	Pagos, transferencias, depósitos y retiros, con eventos de seguridad	pagos-service (con clientes-service para datos de clientes)

3. Propuesta de arquitectura

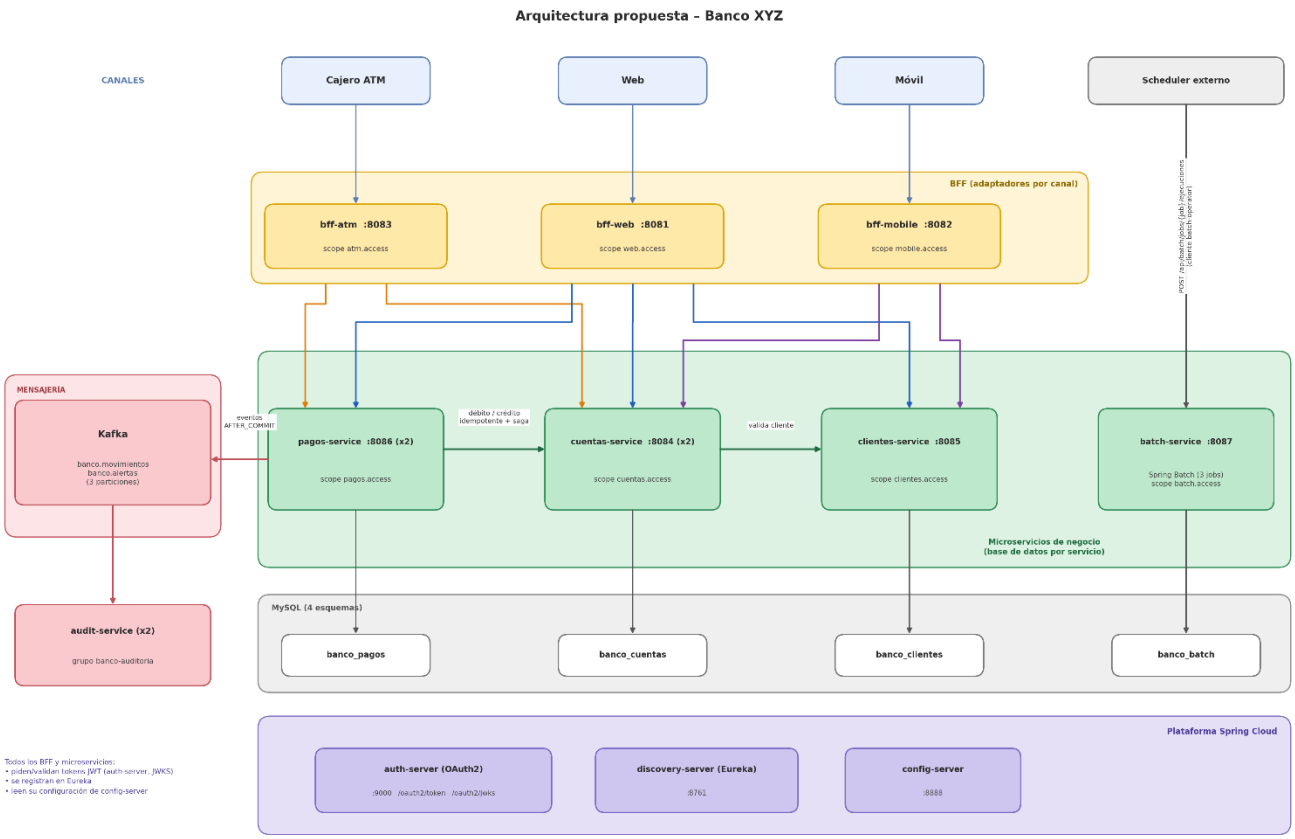


Figura 1. Arquitectura propuesta

3.1 Requerimientos de negocio clave y cómo los resuelve la arquitectura

Requerimiento	Decisión de arquitectura	Justificación
Escalabilidad y mantenibilidad: el banco crece y cada capacidad evoluciona a ritmos distintos	Microservicios por capacidad de negocio con **base de datos por servicio**; descubrimiento con Eureka y balanceo de carga del lado del cliente; configuración centralizada	Cada servicio se despliega y escala solo (cuentas y pagos con 2 réplicas, escalables con *--scale*). Un cambio en pagos no obliga a redesplegar clientes. El legacy solo escala verticalmente
Resiliencia y tolerancia a fallos: un fallo parcial no debe detener la operación bancaria	Resilience4j (Circuit Breaker, Retry, Bulkhead) en cada llamada remota; saga con compensación; operaciones idempotentes; publicación en Kafka después del commit; restart de jobs	Si cuentas cae, pagos responde 503 controlado y deja la transacción reanudable; si Kafka cae, el pago igual se confirma; si la BD cae durante un batch, se reintenta y reanuda
Seguridad distribuida: acceso por canal y servicio con mínimo privilegio	Authorization Server OAuth2 propio; JWT firmado (JWKS); un scope por servicio y por canal; cada servicio es Resource Server	Un token de ATM no accede al BFF web (403). Cada microservicio exige su scope aunque lo llame otro servicio. Sin sesiones ni credenciales compartidas

3.2 Microservicios, responsabilidades y límites

Servicio	Responsabilidad	Datos que posee	Depende de	Scope requerido
clientes-service	Alta, consulta, modificación y baja lógica de clientes; perfil BASICO/PREMIUM/SENIOR	banco_clientes	-	clientes.access
cuentas-service	Apertura, cierre (solo saldo 0), mantenimiento; débito/crédito idempotente con bloqueo pesimista; libro de movimientos	banco_cuentas	clientes-service (valida cliente)	cuentas.access
pagos-service	Pago, transferencia, depósito y retiro; saga con compensación; eventos y alertas Kafka	banco_pagos	cuentas-service, Kafka	pagos.access
batch-service	Tres jobs Spring Batch, reportes CSV, auditoría de rechazos	banco_batch	-	batch.access

Límites: ningún servicio accede a la base de otro; la relación cuenta-cliente es una referencia lógica (clienteId) sin clave foránea entre bases. La librería bank-common contiene solo utilidades técnicas (formato de error, wrapper de resiliencia, cliente de tokens, contrato de eventos) y ninguna regla de negocio, por lo que no acopla los dominios.

3.3 BFF por canal

BFF	Canal / scope	Qué adapta	Servicios que consume
bff-web	web.access	Dashboard completo (cuenta + movimientos), transferencias	cuentas, clientes, pagos
bff-mobile	mobile.access	Respuestas livianas: resumen y movimientos recientes	cuentas, clientes
bff-atm	atm.access	Operaciones de cajero: saldo y retiro (con referencia única para reintentos seguros)	cuentas, pagos

Los BFF no contienen reglas de negocio: autentican al canal (JWT + scope), obtienen su propio token de servicio con los scopes mínimos necesarios, componen y reducen la respuesta y traducen errores. Aceptan HTTPS (server.ssl.*), que en Docker se desactiva solo porque el tráfico es interno a la red del compose.

4. Migración de procesos batch con Spring Batch

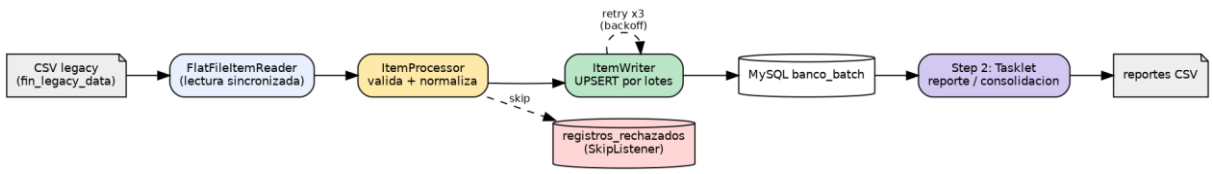


Figura 2. Estructura de cada job

4.1 Diseño común

Cada job tiene un *step* orientado a chunks (lectura con *FlatFileItemReader*, procesamiento con validación y normalización, escritura por lotes con UPSERT) y un *step* final de tipo tasklet que genera el reporte o consolida resultados. Al ser UPSERT, reejecutar un job no duplica datos.

Aspecto	Implementación
Rendimiento	Chunks de 100 registros; jobs 1 y 3 con procesamiento multihilo (4 hilos) y lectura sincronizada; job 2 en un solo hilo porque su regla de duplicados («se conserva el primer registro válido») exige orden
Calidad de datos (skip)	Registros inválidos se omiten y se guardan en *registros_rechazados* con el motivo (SkipListener). Se acepta hasta *batch.skip-limit*; superado el límite el step falla
Fallos transitorios (retry)	Hasta 3 reintentos con espera exponencial (200 ms x2) ante errores transitorios de base de datos
Reejecución automática (restart)	Repositorio de jobs en MySQL. Si un job termina FAILED, *BatchLauncherService* lo reanuda hasta *batch.reejecucion.max-intentos* veces
Disparo	API protegida (*POST /api/batch/jobs/{job}/ejecuciones?dataset=*), apta para un scheduler externo (p. ej. EventBridge)

4.2 Reglas de negocio de cada job

Job	Reglas
reporteTransaccionesDiarias	Fecha en 4 formatos (yyyy-MM-dd, yyyy/MM/dd, dd-MM-yyyy, dd/MM/yyyy); tipo débito|crédito; monto numérico. Se conserva y marca como anomalía el monto negativo (MONTO_NEGATIVO) o cero (MONTO_CERO). Reporte: totales de débitos/créditos, cantidad y anomalías por día
calculoInteresesMensuales	Tipo ahorro|préstamo|hipoteca; saldo obligatorio y no negativo; edad entre 18 y 110 si viene informada; duplicados de cuenta se filtran. Interés mensual = saldo x tasa anual / 12 (ahorro 1,5 %, préstamo 5 %, hipoteca 4 %), redondeo HALF_UP a 2 decimales
estadosCuentaAnuales	Transacción depósito|retiro|compra|pago (se normaliza «depósito» con acento); monto distinto de cero; signo normalizado (depósito positivo, el resto negativo); descripción vacía = «Sin descripción»; clave MD5 evita filas duplicadas. Consolidación por cuenta y año

4.3 Verificación de equivalencia con el legacy

El script independiente *scripts/verificar-equivalencia.py* recalcula los resultados esperados con las mismas reglas. Estos son los valores de referencia contra los que se contrasta cada ejecución de los jobs (la columna «Resultado del job» se completa con la respuesta de *POST /api/batch/jobs/...* y *GET /api/batch/resultados*).

Dataset / job	Leídos	Escritos / válidos	Omitidos	Referencia adicional	Resultado del job
semana_1 / diario	10	10	0	2 anomalías; débitos 5.200; créditos 4.500	completar
semana_1 / intereses	8	8	0	interés mensual total 194,16	completar
semana_1 / estados	9	8	1 (monto cero)	7 cuentas-año; saldo neto 11.400	completar
semana_3 / diario	1000	480	520 (309 tipo, 162 monto vacío, 49 fecha)	88 anomalías; débitos 277.200; créditos 238.600	completar
semana_3 / intereses	1000	50 cuentas	489 (325 tipo, 134 saldo vacío, 30 edad); 461 duplicados filtrados	interés mensual total 1.217,92	completar
semana_3 / estados	1000	835 filas únicas (836 válidas, 1 duplicada)	164 (116 monto cero, 48 vacío)	20 cuentas-año; saldo neto -397.000	completar

El volumen de rechazos de semana_3 (hasta 52 %) es coherente con el objetivo del ejercicio: archivos con errores intencionales que el sistema legacy no gestionaba. Por eso el límite de skips se fijó en 700 por step.

5. Consistencia de datos, resiliencia y mensajería

5.1 Consistencia en un entorno distribuido

Al separar bases de datos no existe una transacción única, por lo que se aplicó un patrón **saga orquestada** en pagos-service, sin commit en dos fases:

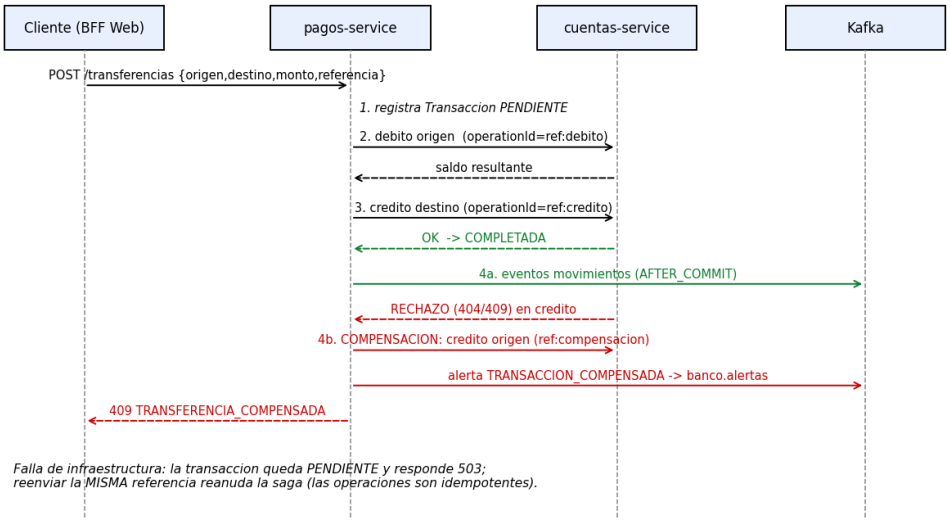


Figura 3. Transferencia: camino feliz, compensación y falla de infraestructura

Mecanismo	Para qué sirve
Idempotencia por *operationId* / referencia	Cada paso remoto (ref:debito, ref:credito, ref:compensacion) se aplica una sola vez aunque se reintente; repetir una referencia devuelve la transacción original
Bloqueo pesimista en cuentas	Dos operaciones concurrentes sobre la misma cuenta se serializan; saldo y movimiento se confirman en una transacción local
Compensación	Si el crédito es rechazado, se devuelve el débito y se emite una alerta; si la compensación falla, la transacción pasa a REVISION_MANUAL con alerta CRITICA
Estado PENDIENTE reanudable	Ante indisponibilidad no se compensa a ciegas (el crédito podría haberse aplicado): se responde 503 y reenviar la misma referencia reanuda la saga
Llamadas remotas fuera de transacciones de BD	No se retienen conexiones mientras se espera a otro servicio

5.2 Resiliencia y comportamientos alternativos

Falla	Comportamiento
pagos -> cuentas caído o lento	Retry (3 intentos) + Circuit Breaker cuentasApi + Bulkhead; fallback: 503 SERVICIO_NO_DISPONIBLE y transacción PENDIENTE
cuentas -> clientes caído	Circuit Breaker clientesApi; alternativa: si el request trae nombre y edad, la cuenta se abre en estado PENDIENTE_VERIFICACION; si no, 503
Kafka caído	Circuit Breaker kafkaPublisher y max.block.ms acotado: el pago ya confirmado no falla; el evento se registra como no publicado (mejora: patrón Outbox)
BFF -> microservicio	Timeouts explícitos, Retry solo en consultas e idempotentes, Circuit Breaker; los errores funcionales (404/409) no abren el circuito
Una réplica caída	Eureka la retira y el LoadBalancer reparte entre las vivas
BD o proceso batch caído	Retry con backoff, restart automático del job y UPSERT idempotente
Contenedor caído	restart: unless-stopped y healthchecks en Docker

Los parámetros de cada circuito (ventana, umbral de fallos, tiempo en estado abierto) se gestionan en config-repo/*.properties vía Spring Cloud Config, sin recompilar.

5.3 Seguridad

Cliente OAuth2	Scopes	Uso
web-app / mobile-app / atm-app	web.access / mobile.access / atm.access	Canales frente a su BFF
bff-web	cuentas.access clientes.access pagos.access	BFF web hacia microservicios
bff-mobile	cuentas.access clientes.access	BFF móvil (solo lectura)
bff-atm	cuentas.access pagos.access	BFF cajero
cuentas-service	clientes.access	Validar clientes
pagos-service	cuentas.access	Debitar y acreditar
batch-operator	batch.access	Disparar los jobs

Todos los servicios validan el JWT (firma vía JWKS, emisor, expiración) y exigen su scope con SecurityFilterChain; los errores 401/403 usan el formato de error común. Limitación declarada: los secretos usan {noop} y valores por defecto de desarrollo; en la nube deben ir en un gestor de secretos con BCrypt.

5.4 Mensajería con Kafka

Tópico	Productor	Consumidor	Contenido
banco.movimientos	pagos-service (después del commit)	audit-service (x2, grupo banco-auditoria)	Transacciones completadas (JSON Schema v1 validado en productor y consumidor, idempotencia por eventId, 3 particiones, clave = cuenta)
banco.alertas	pagos-service	audit-service	Alertas de seguridad: MONTO_INUSUAL (monto >= umbral configurable), TRANSACCION_COMPENSADA, COMPENSACION_FALLIDA

6. Contenerización, escalabilidad y monitorización

Cada módulo tiene un Dockerfile multi-etapa (compilación Maven y ejecución con JRE ligero, usuario no root y HEALTHCHECK). docker-compose.yaml orquesta MySQL (una base por servicio, creadas por script de inicio), Kafka, la plataforma y los servicios con depends_on condicionado a salud. cuentas-service y pagos-service corren con dos réplicas y se escalan con --scale; el balanceo es de lado cliente (Spring Cloud LoadBalancer sobre Eureka), por lo que los microservicios no publican puertos al host y el único borde son los BFF. Los pasos de despliegue en AWS (EC2 + Compose y ECS Fargate + RDS + MSK) están en despliegue.md.

Monitorización: Actuator (/actuator/health, métricas y estado de los circuitos), panel de Eureka, logs estructurados por servicio, tabla de rechazos del batch y alertas de seguridad en Kafka.

Como evolución se propone Prometheus/Grafana y trazabilidad distribuida con OpenTelemetry.

7. Comparación con el sistema legacy

Dimensión	Legacy (COBOL + Shell)	Nueva arquitectura
Escalabilidad	Vertical, limitada al mainframe	Horizontal por servicio (réplicas, Auto Scaling)
Despliegue	Monolítico, ventanas de mantenimiento	Contenedores independientes, cambios por servicio
Integración	Archivos planos y scripts	REST con OAuth2 y eventos Kafka
Seguridad	Perimetral y centralizada	JWT por canal y servicio, mínimo privilegio
Tolerancia a fallos	Un fallo detiene el proceso completo	Circuit Breaker, saga, retry, restart y degradación controlada
Calidad de datos	Registros erróneos sin trazabilidad	Skip + auditoría de cada rechazo con su motivo
Mantenibilidad / talento	COBOL, conocimiento escaso	Java/Spring, pruebas, CI, código versionado

8. Desafíos, soluciones y mejoras futuras

Desafío	Solución
Descomponer el Core sin perder consistencia	Saga con compensación e idempotencia; referencia lógica entre bases
Datos legacy defectuosos y heterogéneos	Validaciones centralizadas, skip con auditoría y límite configurable
Reintentos que duplican dinero	operationId/referencia única por paso
Dependencia de Kafka	Publicación AFTER_COMMIT y Circuit Breaker
Un solo job no reproducible por orden de hilos	Job de intereses en un hilo; resto con UPSERT idempotente

Mejoras: patrón Outbox para eventos, API Gateway como borde único, tracing distribuido, secretos en un gestor y BCrypt, autenticación de usuarios finales (authorization code + PKCE), pruebas de contrato entre servicios, particionado del batch para volúmenes grandes y programación de jobs.

9. Conclusiones

La solución cumple el objetivo académico de presentar microservicios de negocio independientes con responsabilidades delimitadas (clientes, cuentas, pagos y batch), con los BFF como adaptadores de canal, manteniendo la seguridad OAuth2, la resiliencia, la mensajería y la orquestación conseguidas en la entrega anterior, y migrando los tres procesos críticos del legacy a Spring Batch con tolerancia a fallos demostrable.



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC
La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.